

Code

```
# Import necessary libraries
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.datasets import load_breast_cancer
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.metrics import confusion_matrix, classification_report, roc_auc_score,  
roc_curve, precision_recall_curve
```

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
```

```
from tensorflow.keras.callbacks import EarlyStopping, TensorBoard
```

```
from tensorflow.keras.regularizers import l2
```

```
import shap
```

```
import datetime
```

```
# Load and explore the dataset
```

```
data = load_breast_cancer()
```

```
X = data.data
```

```
y = data.target
```

```
# Convert to DataFrame for better visualization
```

```
df = pd.DataFrame(X, columns=data.feature_names)
```

```
df['target'] = y
```

```
# Data overview
```

```
print('Data Shape:', df.shape)
```

```
print('Class Distribution:')
```

```
print(df['target'].value_counts())
```

```
# Enhanced correlation heatmap including all features
```

```
plt.figure(figsize=(16, 12))
```

```
corr_matrix = df.corr()
```

```
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt='.2f', linewidths=0.5)
```

```
plt.title('Enhanced Correlation Matrix - All Features')
```

```
plt.show()
```

```
# Split the data
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# Feature Scaling: Normalizing data using StandardScaler
```

```
scaler = StandardScaler()
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)
```

TensorBoard for training visualization

```
log_dir = 'logs/fit/' + datetime.datetime.now().strftime('%Y%m%d-%H%M%S')
```

```
tensorboard_callback = TensorBoard(log_dir=log_dir, histogram_freq=1)
```

Build the optimized ANN model

```
model = Sequential([  
    Dense(64, activation='relu', kernel_regularizer=l2(0.001), input_dim=X_train.shape[1]),  
    BatchNormalization(),  
    Dropout(0.4),  
    Dense(32, activation='relu', kernel_regularizer=l2(0.001)),  
    BatchNormalization(),  
    Dropout(0.3),  
    Dense(16, activation='relu', kernel_regularizer=l2(0.001)),  
    Dropout(0.2),  
    Dense(1, activation='sigmoid')  
])
```

Compile the model

```
model.compile(optimizer='nadam', loss='binary_crossentropy', metrics=['accuracy'])
```

Early stopping to avoid overfitting

```
early_stopping = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)
```

Train the model

```
history = model.fit(X_train, y_train, batch_size=32, epochs=50, validation_split=0.2,  
callbacks=[early_stopping, tensorboard_callback], verbose=1)
```

Plot training accuracy and loss

```
plt.figure(figsize=(10, 5))  
  
plt.plot(history.history['accuracy'], label='Training Accuracy')  
  
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')  
  
plt.title('Model Accuracy')  
  
plt.xlabel('Epochs')  
  
plt.ylabel('Accuracy')  
  
plt.legend()  
  
plt.show()
```

Evaluate the model

```
loss, accuracy = model.evaluate(X_test, y_test)  
  
print(f'Test Accuracy: {accuracy:.4f}')
```

Predictions

```
y_pred = (model.predict(X_test) > 0.5).astype('int32')
```

Confusion matrix

```
cm = confusion_matrix(y_test, y_pred)  
  
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')  
  
plt.title('Confusion Matrix')  
  
plt.show()
```

ROC Curve

```
y_probs = model.predict(X_test)

fpr, tpr, _ = roc_curve(y_test, y_probs)

roc_auc = roc_auc_score(y_test, y_probs)

plt.plot(fpr, tpr, label=f'AUC = {roc_auc:.2f}')

plt.plot([0, 1], [0, 1], 'k--')

plt.title('ROC Curve')

plt.xlabel('False Positive Rate')

plt.ylabel('True Positive Rate')

plt.legend(loc='lower right')

plt.show()
```

Precision-Recall Curve

```
precision, recall, _ = precision_recall_curve(y_test, y_probs)

plt.plot(recall, precision, marker='.', label='ANN')

plt.title('Precision-Recall Curve')

plt.xlabel('Recall')

plt.ylabel('Precision')

plt.show()
```

SHAP Analysis for feature importance

```
explainer = shap.KernelExplainer(model.predict, X_train)

shap_values = explainer.shap_values(X_test)
```

```
shap.summary_plot(shap_values, features=X_test, feature_names=data.feature_names)
```